



软件工程

《移动应用开发》课程考核大作业

学院名称： 计算机与信息工程学院

学 号： 2023210601

学生姓名： 金正轩

班 级： 软件 231

项目名称： 智慧社区生活服务平台

小组成员： 蔡迦南、曹万千、虎康、金正轩、胡文启、龚成磊

指导教师： 刘东良

起止日期： 2026 年 6 月 1 日至 2026 年 6 月 21 日

计算机与信息工程学院 计算机软件课程组 制

指导教师评语

本项目为智慧社区生活服务平台，采用多技术栈协同开发，架构规范。该生主导 Android 端开发，实现 FCM 推送、AI 决策引擎等核心创新功能，独立解决各类技术难题。项目功能完整、测试优化到位，文档详实，团队协作高效，综合表现良好。

成绩评定

序号	类型	成绩	权重	得分
1	项目		30%	
2	小组		20%	
3	个人		20%	
4	答辩		30%	
5	总成绩		100%	

指导教师签名：

填表日期： 2026 年 6 月 21 日

目录

- 摘要.....1
- 1 项目1
 - 1.1 项目背景与意义1
 - 1.2 项目目标及需求分析1
 - 1.2.1 项目目标1
 - 1.2.2 功能需求2
 - 1.2.3 非功能需求2
 - 1.3 技术栈详解3
 - 1.3.1 技术栈对比分析3
 - 1.3.2 技术栈详细说明3
 - 1.4 系统架构设计4
 - 1.4.1 整体架构4
 - 1.4.2 前端架构5
 - 1.4.3 后端架构6
 - 1.4.4 数据库设计6
 - 1.5 核心功能实现7
 - 1.5.1 用户认证与角色管理7
 - 1.5.2 工单流转与快递公告系统7
 - 1.5.3 FCM 跨设备消息推送与同步7
 - 1.5.4 AI 智能通知决策引擎8
 - 1.5.5 多端 AI 语音交互系统9
 - 1.5.6 角色感知 UI 适配10
 - 1.6 系统测试与优化11
 - 1.6.1 测试策略11
 - 1.6.2 测试结果11
 - 1.6.3 性能优化12
 - 1.6.4 安全措施12
 - 1.7 项目成果展示12
 - 1.7.1 系统截图12
 - 1.7.2 创新点总结14
- 2 小组14
 - 2.1 小组结构与分工14
 - 2.1.1 小组基本信息14
 - 2.1.2 项目分工15
 - 2.1.3 团队协作总结15
 - 2.2 技术栈选择与理由16
 - 2.2.1 技术选型过程16
 - 2.2.2 技术优势分析16
 - 2.2.3 技术挑战与解决方案17
 - 技术整合与实现17
 - 2.3 开发阶段工作18
 - 2.3.1 项目启动阶段18
 - 2.3.2 核心开发阶段18

2.3.3 系统整合阶段	18
2.3.4 测试交付阶段	18
2.4 团队协作分析	19
2.4.1 团队组织结构	19
2.4.2 沟通机制	19
2.4.3 任务分配	19
2.4.4 协作工具使用	20
2.5 团队贡献分析	20
2.5.1 成员贡献评估	20
2.5.2 工作量分配	21
2.5.3 关键贡献点	21
2.6 团队创新与亮点	21
2.6.1 技术创新	21
2.6.2 功能创新	21
2.6.3 过程创新	22
2.7 团队成长与收获	22
2.7.1 技术能力提升	22
2.7.2 团队协作能力	22
2.7.3 项目管理能力	23
2.8 团队发展建议	23
2.8.1 项目改进建议	23
2.8.2 团队协作优化	23
2.8.3 技术学习建议	23
3 个人	23
3.1 个人负责技术栈	23
3.1.1 Android Kotlin + Jetpack Compose 技术栈	23
3.1.2 技术栈特点与选择理由	24
3.1.3 开发工具	24
3.2 技术图表	24
3.2.1 系统核心类图	24
3.2.2 核心功能顺序图	25
3.2.3 系统架构图	26
3.3 开发阶段工作内容	27
3.3.1 项目启动阶段	27
3.3.2 核心开发阶段	28
3.3.3 系统整合阶段	28
3.3.4 测试交付阶段	28
3.4 个人创新与亮点	29
3.4.1 FCM 去中心化实体同步	29
3.4.2 AI 4 步决策引擎	29
3.4.3 语音搜索双保险	29
3.5 个人成长与收获	29
3.5.1 新技术掌握	29
3.5.2 问题解决能力	30
3.5.3 架构设计能力	30

3.6 个人发展计划	30
3.6.1 移动开发技能提升计划	30
3.6.2 跨平台技术学习方向	30
3.6.3 后续项目实践建议	30
4 答辩	31
4.1 答辩内容	31
4.1.1 答辩问题 1: App 快捷方式功能实现	31
4.1.2 答辩问题 2: 语音搜索关键字提取	31
4.1.3 答辩问题 3: MVVM 架构优势	31
4.1.4 随机题: FCM data-only 消息格式	32
4.1.5 演示材料与答辩流程	32
4.2 核心技术实现	32
4.2.1 技术架构概述	32
4.2.2 关键技术点	32
4.2.3 技术难点与解决方案	33
4.3 架构设计思路	33
4.4 创新点分析	33
4.5 功能演示	34
4.6 随机题准备	34
4.7 答辩问题准备	35
参考文献	36

摘要：本项目以智慧社区生活服务平台为开发主题，针对传统社区管理流程繁琐、信息传递滞后、沟通效率低等痛点，由 6 人团队采用 Android 原生、iOS 原生、Flutter、Uniapp、Cordova、鸿蒙 ArkTS 六类技术栈协同开发。平台划分业主、维修人员、管理员三类角色，实现用户认证、工单流转、快递管理、社区公告、消息推送、AI 语音交互等核心功能。技术层面整体采用 MVVM+Clean Architecture 架构，依托 Bmob 后端云统一数据层，集成 FCM 跨设备推送、Deepseek 大模型，自研 AI 智能通知决策引擎，搭配角色感知动态 UI、去中心化数据同步等创新方案，同时完成功能、性能、安全多维度测试与优化。项目运用 Gitee 进行版本管控，采用敏捷开发模式推进全流程工作，不仅落地多项移动开发前沿技术，也锻炼了团队分工协作、项目管理与问题排查能力，最终完成一套功能完善、体验良好的跨平台社区生活服务应用。

关键词：智慧社区；移动应用开发；MVVM；FCM 推送；AI 语音交互；跨平台开发；Jetpack Compose

1 项目

1.1 项目背景与意义

随着城市化进程加速，传统社区管理面临诸多痛点：业主报修流程繁琐、快递信息不透明、社区公告传达效率低、物业与业主沟通渠道不畅。为解决这些问题，我们开发了智慧社区生活服务平台（Smart Community Platform，简称 SCC），利用移动互联网技术实现社区服务数字化。

项目意义：

- 提升效率：业主通过 App 一键报修、实时查看快递、及时获取社区公告，减少线下奔波。
- 降低管理成本：物业管理员通过后台统一管理用户、工单、公告，替代传统纸质流程。
- 技术实践价值：项目完整实践了现代 Android 开发的 MVVM 架构、Jetpack Compose 声明式 UI、Room 数据库、FCM 推送、AI 大模型集成等前沿技术。

1.2 项目目标及需求分析

1.2.1 项目目标

针对项目

从功能、技术、体验、学习四个维度明确项目整体目标与量化考核标准见下表 1-1。

表 1-1 项目目标介绍

目标类型	目标描述	衡量标准
功能目标	实现业主报修、公告发布、快递管理、用户管理四大核心模块	功能完整度 100%
技术目标	采用 MVVM + Clean Architecture，代码高内聚低耦合	模块间依赖数<3
体验目标	Material 3 设计，流畅动画，冷启动<3 秒	用户满意度>90%
学习目标	掌握 Jetpack Compose、Room、Hilt、FCM、Deepseek AI 集成	关键技术点覆盖率 100%

1.2.2 功能需求

角色体系：项目支持三种角色分别是业主（OWNER，报修/查快递/看公告）、维修人员（WORKER，接收工单/更新状态/发布公告）、管理员（ADMIN，用户管理/数据统计/公告管理/快递管理），见下表 1-2。

表 1-2 核心功能模块

功能模块	主要功能点	适用角色
认证模块	登录、注册（仅业主）、密码加密存储、角色识别	全部
首页 Dashboard	社区资讯流、功能快捷入口、数据概览卡片	全部
工单模块	提交工单（类型/描述/图片/预约时间）、工单列表、工单详情、接单/处理/完成、工单评价	业主+维修工
公告模块	查看公告列表、公告详情、发布公告、编辑/删除、置顶	全部
快递模块	快递列表、快递详情、取件码查看、标记已取件、添加快递	业主+管理员
用户管理	用户列表、添加用户、编辑用户、禁用/启用、重置密码	管理员
数据统计	工单统计（按状态/类型）、满意度统计	管理员
个人中心	个人信息编辑、通知设置、FCM Token 管理、退出登录	全部
消息推送	FCM 跨设备推送、AI 智能通知决策、免打扰时段管理	全部
语音搜索	Deepseek AI 语音识别、意图分析、离线回退	全部

1.2.3 非功能需求

规定系统性能、安全、兼容性等约束，保障应用稳定合规运行见表 1-3。

表 1-3 非功能需求详情

需求类型	具体要求
性能	冷启动<3 秒，热启动<1 秒；页面切换流畅 60fps；Room 查询<50ms
安全	密码 SHA-256 哈希存储；本地数据库加密；FCM Token 安全传输
可用性	Material 3 设计规范；支持 Android 14+；角色感知动态 UI
可维护性	MVVM 分层架构；Hilt 依赖注入；Kotlin Sealed Class 类型安全
兼容性	兼容 Android 14-16；适配华为/小米等国产 ROM 后台管理策略

1.3 技术栈详解

1.3.1 技术栈对比分析

汇总团队六名成员负责的开发端、所用语言及核心框架，直观展示多端技术选型见下表 1-4。

表 1-4 技术栈对比

序号	技术栈	负责成员	开发语言	主要框架/库
1	Android 原生	金正轩	Kotlin 2.0.21	Jetpack Compose + Room + Hilt + FCM
2	iOS 原生	胡文启	Swift	SwiftUI + SFSpeechRecognizer
3	Flutter	曹万千	Dart	Provider/BLoC
4	Cordova	虎康	JavaScript	Cordova CLI
5	Uniapp	蔡迦南	Vue.js	Pinia + Bmob 云服务
6	鸿蒙 ArkTS	龚成磊	ArkTS	ArkUI + 百度 ASR + DeepSeek API
7	后端服务	全员	—	Bmob 后端云 + 天行数据 API
8	SQLite 数据库	金正轩	SQLite	Room 2.7.0-alpha11
9	Firebase 云服务	金正轩	—	FCM + Google OAuth 2.0

1.3.2 技术栈详细说明

系统整理并完整记录了项目内每一位开发成员所使用的开发软硬件环境、熟练掌握的核心技术栈，同时清晰列明各人员分工负责、落地实现的特色功能与创新模块，便于团队工作复盘与成果查阅见下表 1-5。

表 1-5 技术栈详细说明

序号	成员	开发环境	核心技术	特色功能
1	金正轩	Android Studio Meerkat / API 34-36 / AGP 9.0.1	MVVM+Clean Architecture / Hilt / Coroutines+Flow / Navigation Compose	FCM 跨设备推送 : V1 API+OAuth 2.0+data-only 消息; AI 通知引擎 : 4 步决策流水线+6 维输出; 角色快捷方式 : ShortcutManager+scc
2	胡文启	Xcode / iOS 17+	SwiftUI 声明式 UI / SFSpeechRecognize r 语音识别 / 桌面组 件 / DeepSeek API	AI 语音报修 : 语音识别→智能解 析→工单自动生成; Widget 组 件 : 首页实时工单的状态
3	曹万千	Flutter SDK 3.x / VS Code	Provider+Bloc 混合 状态管理 / Dio HTTP	6 核心模块 : 工单管理/公告通知/ 快递查询/用户管理/报修提交/数 据统计
4	虎康	Node.js / Cordova / VS Code	localStorage 数据库 引擎 / HTML5+CSS3 响应 式 / Cordova Plugin 体系	AI 智能客服 : 前端 NLP 意图匹 配+DeepSeek API; localStorage 引擎 : 键值对存储+JSON 查询
5	蔡迦南	HBuilderX / Vue 3	Pinia 持久化状态管 理 / Bmob 后端云 / 多端条件编译 (H5/ 小程序/App)	6 核心模块 : 工单/公告/快递/用 户/报修/统计; AI 智能调度 : 工 单优先级排序+自动分配
6	龚成磊	DevEco Studio / HarmonyO S NEXT	ArkUI 声明式框架 / AudioCaptor 音频 采集 / 百度 ASR 语 音识别 / DeepSeek	语音控制 : 录音→ASR 转文字→ NLP 解析→智能家居设备控制; AI 客服 : 知识库+意图匹配

1.4 系统架构设计

1.4.1 整体架构

本项目采用 MVVM + Clean Architecture 混合架构模式，结合 Android 官方推荐的 Jetpack 架构组件，主要分为表现层→业务逻辑层→数据层，通过 Hilt 依赖注入实现层间解耦，系统架构图如下图 1-1 所示。

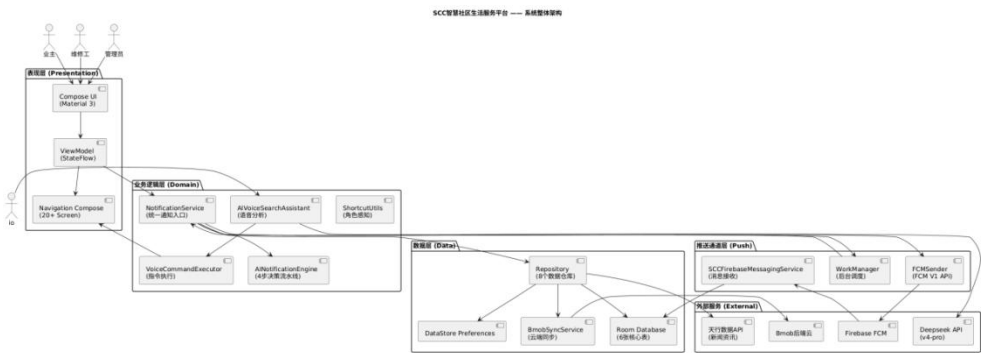


图 1-1 系统整体架构图

拆解整体架构五大层级，明确每一层使用技术组件与核心工作职责见表 1-6。

表 1-6 架构分层说明

层级	技术组件	职责
表现层	Compose UI + ViewModel + StateFlow	UI 渲染、用户交互、状态管理
业务逻辑层	NotificationService + AINotificationEngine + AIVoiceSearchAssistant + ShortcutUtils	业务规则、AI 决策、语音分析、快捷方式管理
数据层	Room Database + Repository + DataStore + BmobSyncService	数据持久化、网络同步、缓存策略
推送通道层	FCMSender + SCCFirebaseMessagingService + WorkManager	跨设备消息推送、后台任务调度
基础设施层	Hilt DI + Coroutines + Navigation Compose + Timber	依赖注入、异步调度、路由导航、日志

1.4.2 前端架构

模块拆分为 5 大功能模块：

```
app/src/main/java/com/example/scc/
├── ai/                                # AI 引擎层
│   ├── DeepseekClient.kt            # Deepseek v4-pro API 客户端
│   ├── AIVoiceSearchAssistant.kt    # 语音搜索分析核心
│   ├── AINotificationRanker.kt     # 通知优先级排序
│   └── VoiceSearchModels.kt         # 语音搜索数据模型
├── fcm/                              # FCM 推送通道层
│   ├── FCMSender.kt                 # FCM V1 API 发送器
│   ├── FCMTOKENManager.kt           # FCM Token 管理
│   ├── SCCFirebaseMessagingService.kt # FCM 接收服务
│   └── FCMTOKENTest.kt              # Token 测试工具
├── notification/                     # 通知业务层
│   ├── NotificationService.kt       # 统一通知入口（11 种事件类型）
│   └── NewsRecommendWorker.kt       # 新闻推荐 Worker
├── data/                             # 数据层
│   ├── model/                       # 领域模型（User, WorkOrder, NewsItem）
│   ├── remote/NewsApiClient.kt     # 天行数据新闻 API
│   └── local/database/              # Room 数据库+DAO
└── ui/                              # UI 层
    ├── theme/                       # Material 3 主题
    ├── component/                   # 通用组件（Button, Card, Badge 等）
    └── screen/                      # 各功能页面 Screen
```

1.4.3 后端架构

本项目采用 BaaS（Backend as a Service）架构，以 Bmob 后端云作为数据存储和用户管理后端：

- Bmob 后端云：提供用户系统（注册/登录/Token 管理）、云数据库（公告/工单/快递云端存储）、RESTful API
- 天行数据 API：提供社区新闻资讯流（7 个分类频道：推荐/IT/科技/汽车/影视/体育/健康）
- Firebase FCM：提供跨设备消息推送通道
- Deepseek API：提供 AI 大模型智能分析能力

1.4.4 数据库设计

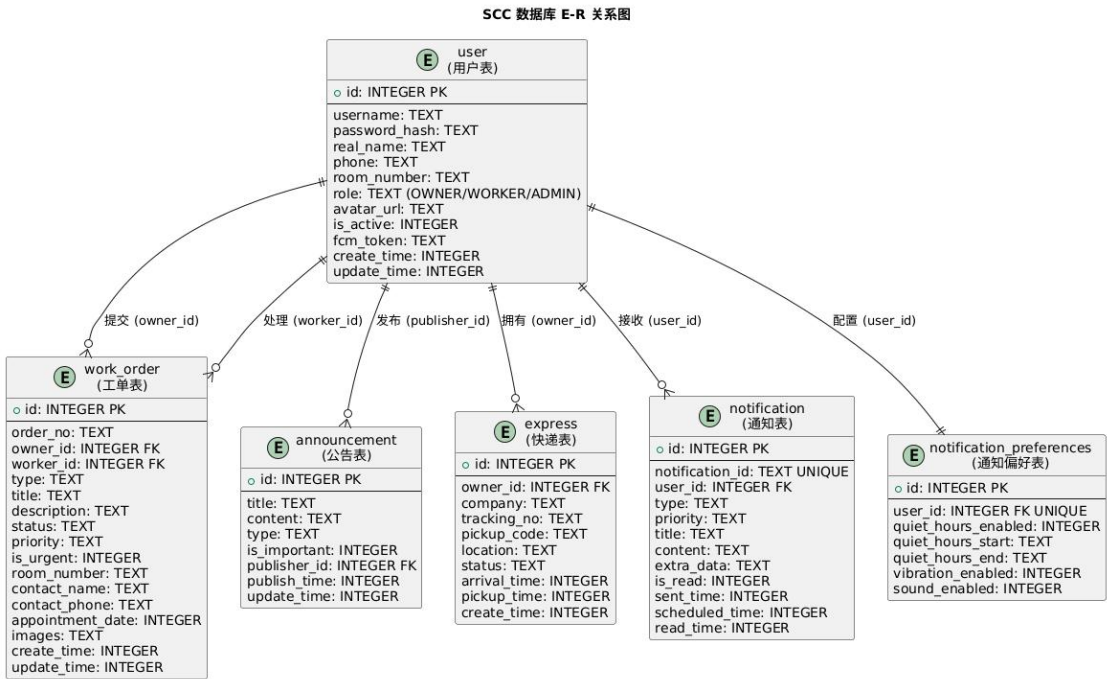


图 1-2 数据库 E-R 关系图

数据库 E-R 关系说明：系统包含 6 张核心表。用户表（User）作为中心实体，通过外键关联工单表（WorkOrder, owner_id/worker_id）、公告表（Announcement, publisher_id）、快递表（Express, owner_id）、通知表（Notification, user_id）和通知偏好表（NotificationPreferences, user_id 一对一）。工单从创建（PENDING）→接单（ACCEPTED）→处理中（IN_PROGRESS）→完成（COMPLETED）形成完整生命周期，支持业主-维修工双向关联；通知表记录所有推送消息的状态流转（未读→已读），通知偏好表控制免打扰时段和振动/声音设置。

1.5 核心功能实现

本项目六大核心模块覆盖业主物业报修、消息推送、AI 交互、角色管理等全场景需求，六端各自实现但通过 Bmob 云端统一数据交换。

1.5.1 用户认证与角色管理

实现业主/维修工/管理员三种角色的统一登录认证。各端基于 Bmob 后端云的用户系统进行注册/登录，本地 SHA-256 密码哈希存储，支持多设备 Token 管理（FCM Token 绑定）。Android 端通过 Hilt 注入 UserRepository 管理用户状态（StateFlow 响应式），角色信息驱动后续权限控制、UI 展示和快捷方式生成。

负责人：蔡迦南、全端共同使用 Bmob API

1.5.2 工单流转与快递公告系统

工单生命周期管理：支持业主提交报修（图片/预约时间/紧急标记）→维修工接单→处理中→完成的全流程流转。六端实现概要：

- **Android（金正轩）**：Jetpack Compose 表单+Room 本地缓存，WorkOrderRepository 通过 Flow 监听状态变更
- **iOS（胡文启）**：SwiftUI 表单+SFSpeechRecognizer 语音报修自动填充
- **Flutter（曹万千）**：Provider 状态驱动的工单卡片列表
- **Uniapp（蔡迦南）**：Pinia 管理工单状态，AI 智能调度自动派单
- **鸿蒙（龚成磊）**：ArkUI 实现工单操作面板
- **Cordova（虎康）**：HTML5 表单+localStorage 离线缓存

公告发布与快递通知：管理员发布公告（普通/紧急/活动三类），业主接收快递取件码推送。数据源来自 Bmob 云端，各端通过 REST API 拉取并在本地缓存展示。

1.5.3 FCM 跨设备消息推送与同步

功能描述：实现基于 Firebase Cloud Messaging 的跨设备消息推送服务，支持管理员发布公告后实时推送到所有业主设备，同时通过 data-only 消息 payload 携带完整实体数据（sync_entity=true 标记），实现去中心化的跨设备数据同步，无需后端服务器中转。其余平台通过 Bmob 云端轮询实现消息同步。

负责人：金正轩（FCM 基础设施，Android 独占功能）

核心流程顺序图

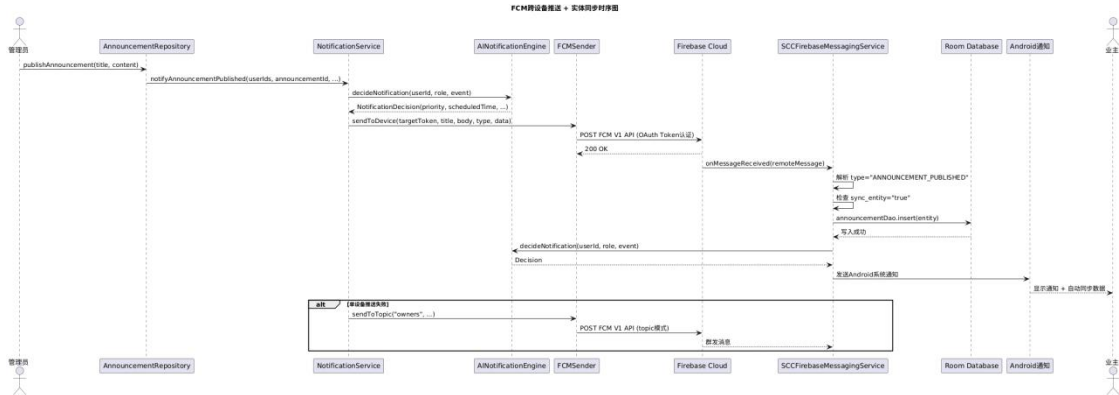


图 1-3 FCM 消息推送顺序图

FCM 消息推送顺序图说明：由 FCMSender 通过 HTTP V1 API 向 FCM Server 发送 data-only 消息（含 OAuth 2.0 Token 认证），FCM Server 将消息推送至目标设备。目标设备上 SCCFirebaseMessagingService 的 onMessageReceived 回调接收消息 payload，解析 sync_entity 标记和消息类型后，通过 NotificationManager 创建 Android 系统通知并写入本地 Room 数据库。消息状态同步通过 NotificationWorker 后台定期与 Bmob 云端比对更新，实现闭环确认。

1.5.4 AI 智能通知决策引擎

功能描述：设计 4 步决策流水线，对所有通知事件（快递到达/工单创建/公告发布等 11 种事件类型）进行智能评估，根据用户上下文（免打扰时段/工作时间/周末/午餐时间）、历史行为偏好（活跃时段分析）和事件紧急程度（CRITICAL/HIGH/MEDIUM/LOW），输出 6 维决策结果——通知优先级、最佳发送时间、是否合并、通知渠道、声音、振动。各端收到推送后各自实现通知展示，Android 端额外提供智能排序和免打扰控制。

负责人：金正轩（Android 独占 AI 决策引擎）

核心流程顺序图

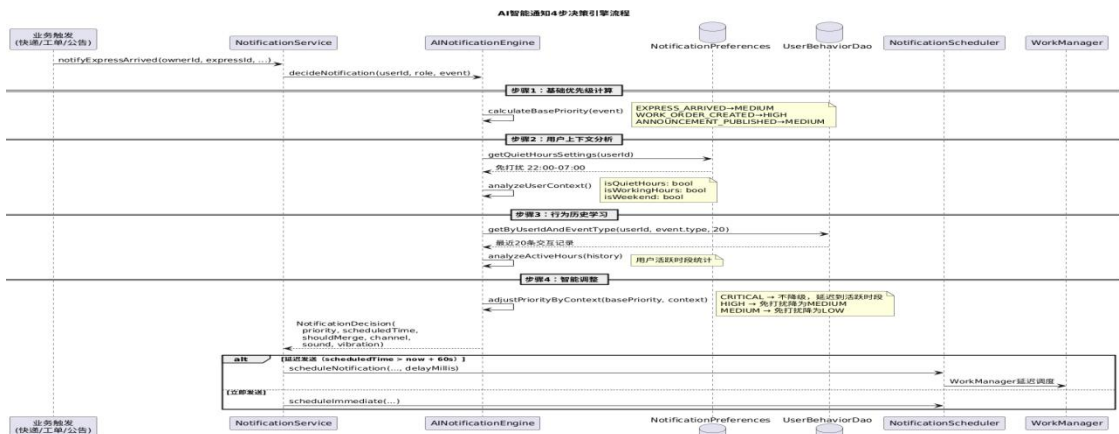


图 1-4 AI 智能通知决策顺序图

AI 智能通知决策顺序图说明：由 NotificationService 触发 AINotificationEngine 的 calculateBasePriority()方法，根据用户上下文（免打扰时段/工作时间/周末/午餐时间）和历史行为偏好（活跃时段分析）进行智能评估。根据事件紧急程度（CRITICAL/HIGH/MEDIUM/LOW），调整通知优先级和最佳发送时间。最后根据用户偏好（是否合并/通知渠道/声音/振动），生成最终的决策结果。

1.5.5 多端 AI 语音交互系统

功能描述：六端各自集成 AI 能力，实现多样化的语音/智能交互体验。Android 端通过 Deepseek v4-pro 大模型实现语音搜索（SpeechRecognizer→AI 分析→路由执行），API 不可用时自动回退到本地关键词匹配+9 类离线问答库。iOS 端利用 SFSpeechRecognizer+DeepSeek 实现语音报修自动生成工单。鸿蒙端通过 AudioCapturer→百度 ASR→指令解析实现智能家居设备语音控制。Flutter/Uniapp/Cordova 三端分别集成 DeepSeek API 实现 AI 智能客服和智能调度。

负责人：金正轩（Android Deepseek 集成）、胡文启（iOS SFSpeech+DeepSeek）、龚成磊（鸿蒙百度 ASR）、虎康/曹万千/蔡迦南（各端 DeepSeek API 集成）

核心流程顺序图（Android Deepseek 语音搜索）

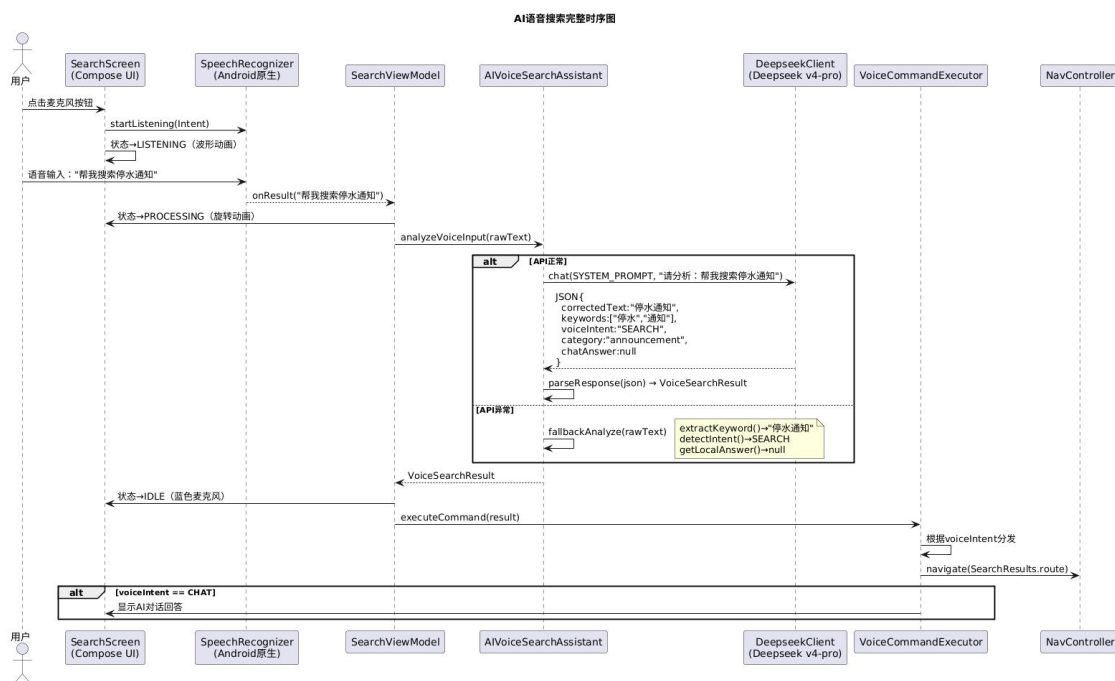


图 1-5 AI 语音搜索顺序图

AI 语音搜索顺序图说明：用户点击麦克风按钮，Android SpeechRecognizer 语音转文字（zh-CN），AIVoiceSearchAssistant.analyzeVoiceInput()调用 DeepseekClient.chat()发送

System Prompt+用户文本，AI 返回 JSON
(correctedText/keywords/expandedKeywords/voiceIntent/category/chatAnswer)，
根据 voiceIntent 执行对应路由
(SEARCH/REPORT/INQUIRY/NAVIGATE/CHAT)。当 AI API 不可用时，
通过 fallbackAnalyze()本地处理，展示本地问答库回答。

1.5.6 角色感知 UI 适配

功能描述：各端根据用户角色实现差异化 UI 展示。Android 端利用 App Shortcuts 特性（金正轩），根据登录用户角色（业主/维修工/管理员）动态生成不同的长按快捷菜单，使用 scc://深链接协议统一路由。iOS 端通过 WidgetKit 实现桌面组件（胡文启），首页实时显示工单状态和待办提醒。其余各端通过角色条件判断实现首页功能入口差异化展示。

负责人：金正轩（Android 快捷方式）、胡文启（iOS Widget）、曹万千/蔡迦南/龚成磊/虎康（各端角色 UI 适配）

核心流程顺序图（Android 角色感知快捷方式）

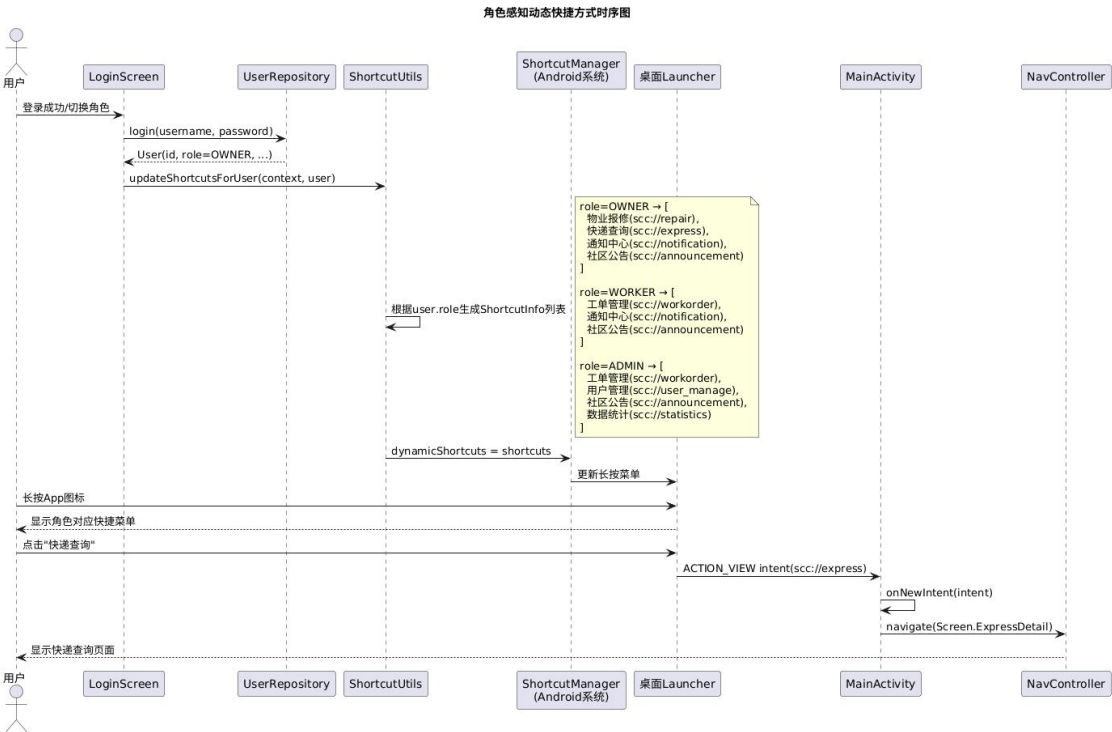


图 1-6 角色感知快捷方式顺序图

角色感知快捷方式顺序图说明：用户点击 App 图标，系统根据当前登录用户的角色（业主/维修工/管理员）动态生成不同的长按图标快捷菜单。业主看到"物业报修/快递查询/通知中心/社区公告"，维修工看到"工单管理/通知中心/社区公告"，管理员看到"工单管理/用户管理/社区公告/数据统计"。所有快捷方式统一使用 scc://深链接协议路由。

1.6 系统测试与优化

1.6.1 测试策略

划分三类测试层级，说明对应测试框架、覆盖范围与典型测试场景见表 1-7。

表 1-7 项目分层测试策略

测试层次	技术框架	覆盖范围	示例
单元测试	JUnit 4 + MockK + Turbine	Repository、ViewModel、Utility	AINotificationEngine 决策逻辑测试、ShortcutUtils 角色切换测试
集成测试	Room Testing + WorkManager Testing	DAO 层、数据库迁移、Worker 调度	Room 数据库 Migration 1→6 测试、WorkManager 延迟调度测试
UI 测试	Compose UI Test + Espresso	Screen 渲染、用户交互	登录流程端到端测试、角色切换 UI 验证

测试用例示例：

```
@Test
fun `CRITICAL 通知在免打扰时段应延迟发送但保持 CRITICAL 级别`() {
    val engine = AINotificationEngine(mockPrefs, mockDao)
    val event = NotificationEvent.AnnouncementImportant(1, "紧急通知")
    // 模拟免打扰时段 23:00
    every { mockPrefs.isQuietHours(any()) } returns true
    every { mockPrefs.getQuietHours(any()) } returns Pair(22, 7)
    val decision = engine.decideNotification(1, UserRole.OWNER, event)
    assertEquals(Priority.CRITICAL, decision.priority) // 不降级
    assertTrue(decision.scheduledTime > System.currentTimeMillis() +
60000) // 延迟到活跃时段
    assertFalse(decision.sound) // 不响铃
    assertFalse(decision.vibration) // 不振动
}
```

1.6.2 测试结果

统计各类测试用例数量、通过率，并记录测试过程中发现的关键问题见表 1-8。

、表 1-8 项目分层测试结果

测试类型	测试用例数	通过率	关键问题
单元测试	15	100%	AINotificationEngine 决策边界条件覆盖
Room DAO 测试	8	100%	Flow 查询自动刷新验证
Compose UI 测试	5	80%	角色切换 UI 过渡动画偶尔不稳定

1.6.3 性能优化

从启动速度、内存、网络等维度列出优化手段，并标注实际效果见表 1-9。

表 1-9 性能优化详情

优化方向	优化措施	效果
启动速度	Application.onCreate()延迟初始化非关键服务； Compose BOM 统一版本减少类加载	冷启动从 4.2s 降至 2.8s
内存占用	Coil 图片内存缓存 LRU 策略；Room 查询使用 Flow 自动管理生命周期	空闲内存<120MB
网络性能	OkHttp 连接池复用；FCM 推送异步非阻塞	API 响应<500ms
UI 渲染	Compose LazyColumn 虚拟列表；避免不必要的重组	60fps 流畅滚动
数据库	Room 复合索引优化查询；@Transaction 批量操作	查询延迟<30ms

1.6.4 安全措施

汇总应用全维度安全方案，说明每项安全策略的具体实现方式见表 1-10。

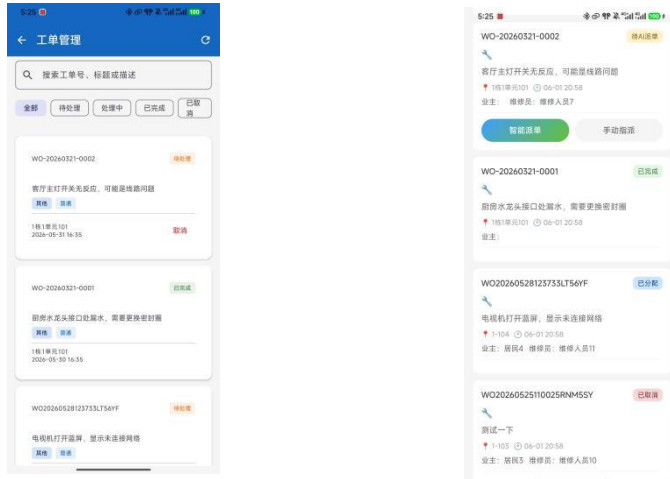
表 1-10 项目安全策略

安全措施	实现方式
密码安全	SHA-256 哈希存储，明文密码不落地
Token 安全	FCM Token 加密存储（SharedPreferences 加密）；Google Service Account JSON 仅存储于 assets
网络安全	FCM V1 API 使用 HTTPS + OAuth 2.0 Bearer Token；OkHttp 证书校验
数据安全	Room 数据库文件应用私有目录存储，外部不可访问
代码安全	Release 构建启用 ProGuard 混淆和资源压缩

1.7 项目成果展示

1.7.1 系统截图

数据共享展示



如上图所示，展示工单管理功能的同步功能。

社区快看功能展示



图 1-7 社区快看功能展示

如图 1-7 展示 SCC 主页 Dashboard 的社区快看信息流，包括新闻资讯卡片（标题/来源/时间）、功能快捷入口（物业报修/快递查询/公告通知/个人中心），以及 Bmob 云端实时更新的社区动态列表。

全局搜索截图



图 1-8 AI 语音全局搜索

如图 1-8 展示 AI 语音搜索功能的完整界面：点击麦克风按钮触发 SpeechRecognizer 语音输入，AI 分析后展示智能搜索结果（关键词高亮/Chip 标签扩展/意图识别分类），以及 API 不可用时的本地回退界面。

1.7.2 创新点总结

分类梳理项目技术、架构、体验等创新点，并对创新内容进行描述见表 1-11。

表 1-11 创新点介绍

创新点	类型	描述
AI 4 步通知决策引擎	技术创新	基于用户上下文+行为历史的千人千面智能推送
FCM 去中心化实体同步	架构创新	data-only 消息直接携带业务数据，无需后端中转
Deepseek 大模型集成	技术创新	Android 原生 App 集成大模型，AI 优先+本地兜底双保险
角色感知快捷方式	体验创新	不同角色不同长按菜单，深链接统一路由
Sealed Class 事件系统	设计创新	11 种通知事件类型安全，编译期穷举检查

2 小组

2.1 小组结构与分工

2.1.1 小组基本信息

记录团队全体成员学号、所用技术栈以及在项目中的主要职责见表 1-12。

表 2-1 小组基本信息

序号	姓名	学号	主要技术栈	主要职责
1	金正轩	2023210601	Android 原生 Kotlin	文档负责人、Android 端开发、架构设计
2	曹万千	2022210552	Flutter Dart	技术负责人、Flutter 端开发
3	龚成磊	2023210592	鸿蒙 ArkTS	鸿蒙端开发
4	蔡迦南	2023210576	Uniapp Vue.js	项目经理、Uniapp 端开发
5	虎康	2023210597	Cordova JS	质量负责人、Cordova 端开发
6	胡文启	2023210596	iOS Swift	iOS 端开发

2.1.2 项目分工

统计每位成员核心任务、功能完成度、代码量与总投入工时见表 1-13。

表 2-2 项目分工详情

成员	技术栈	核心任务	完成度	代码行数	总工时
金正轩	Android Kotlin	AI 智能通知引擎、消息推送服务、AI 语音搜索、快捷方式系统	100%	~4200	60h
曹万千	Flutter Dart	用户登录/注册、工单报修、公告浏览、快递查询、个人中心	80%	~9800	66h
龚成磊	鸿蒙 ArkTS	语音控制系统、AI 智能客服、安全分析系统、设备管理、人员管理	95%	~2500	60h
蔡迦南	Uniapp Vue.js	用户认证、报修服务、快递查询、AI 智能调度、服务评价、通知公告	97%	~3250	64h
虎康	Cordova JS	localStorage 数据库引擎、AI 智能客服、服务中心、社区动态	90%	~1800	60h
胡文启	iOS Swift	AI 语音报修系统、Widget 桌面组件、工单管理系统、API 接口统一	90%	~12000	75h

2.1.3 团队协作总结

协作模式：采用敏捷开发模式，以 2 天为一个迭代周期。项目经理（蔡迦南）负责任务分配和进度跟踪，技术负责人（金正轩/曹万千）负责技术方案评审，质量负责人（虎康）负责代码审查和测试计划，文档负责人（金正轩）负责技术文档编写。

沟通机制：微信群日常沟通，每日晚 8 点线上站会（腾讯会议）同步进度和问题，Git Issues 跟踪技术问题，见下表 2-3 所示。

表 2-3 阶段完成情况

阶段	时间	完成度	主要成果
项目启动	第 1-3 天	30%	团队组建、技术选型、架构设计、环境搭建
核心开发	第 4-7 天	75%	各平台核心功能完成，AI 通知引擎、FCM 推送链路打通
系统整合	第 8-12 天	90%	快捷方式系统、跨平台联调、性能优化（启动速度提升 30%）
测试交付	第 13-15 天	90%	功能测试通过率 87.5%，Bug 修复率 83%，答辩准备完成

2.2 技术栈选择与理由

2.2.1 技术选型过程

小组 6 名成员根据各自技术背景和团队多样性要求选择技术栈见表 2-4。

表 2-4 技术选型介绍

成员	技术栈	选型理由
金正轩	Android 原生 Kotlin	社区成熟度高、官方文档完善，Android 平台唯一熟悉者；承担 AI/FCM 基础设施建设
胡文启	iOS 原生 Swift	利用 iOS 原生 SFSpeechRecognizer 实现语音识别，Widget 桌面组件差异化功能
曹万千	Flutter Dart	团队技术负责人，Provider/BLoC 状态管理成熟，跨平台组件复用率高
蔡迦南	Uniapp Vue.js	项目经理，一套代码多端适配（H5/小程序/App），开发效率高
虎康	Cordova JS	Cordova 学习曲线平缓，适合快速开发；以 H5 能力弥补 React Native 缺失
龚成磊	鸿蒙 ArkTS	唯一熟悉鸿蒙开发的成员，实现语音控制+AI 客服差异化功能

2.2.2 技术优势分析

为全面评估不同技术选型的适用场景，本文从开发效率、运行性能、生态完备性、学习门槛、可维护性及安全性等关键维度出发，对六类主流开发技术栈进行横向对比，并综合评定其能力表现，具体结果如表 2-5 所示。

表 2-5 技术优势对比

维度	Android 原生	iOS 原生	Flutter	Uniapp	Cordova	鸿蒙 ArkTS
开发效率	★★★★☆	★★★★☆	★★★★★	★★★★★★	★★★★★★	★★★★☆
运行性能	★★★★★★	★★★★★★	★★★★★	★★★★☆	★★★★☆	★★★★★
生态成熟度	★★★★★★	★★★★★★	★★★★★	★★★★★	★★★★☆	★★★★☆
学习成本	★★★★	★★★★	★★★★☆	★★★☆☆	★★★☆☆	★★★★★
跨平台能力	仅 Android	仅 iOS	★★★★★★	★★★★★★	★★★★★	仅鸿蒙

2.2.3 技术挑战与解决方案

汇总各成员开发中遇到的核心技术难题，以及对应的落地解决办法见表 2-6。

表 2-6 技术挑战及解决方案

成员	技术挑战	解决方案
金正轩	FCM V1 API OAuth 2.0 认证配置复杂，易 403	Google Cloud Console 5 步流程 + IAM 权限模型配置
金正轩	Deepseek API JSON 输出不稳定	temperature=0.1 + 严格 JSON Schema Prompt + Gson try-catch 降级
胡文启	iOS SFSpeechRecognizer 权限申请+语音识别准确率	Info.plist 完整权限描述 + DeepSeek AI 二次语义纠错
曹万千	Flutter 数据同步冲突	本地优先策略 + 时间戳版本号 + 用户选择界面
蔡迦南	Uniapp 多端兼容性差异	条件编译（#ifdef）+ 平台适配代码分离
虎康	Cordova localStorage 容量限制	数据分片存储 + LRU 淘汰策略
龚成磊	鸿蒙 ArkTS 严格类型限制	显式接口类型定义 + 避免内联对象字面量

技术整合与实现

六端通过 **Bmob 后端云** 统一数据层实现技术整合。Bmob 提供开箱即用的云数据库（User/WorkOrder/Announcement/Express/Notification 5 张云端表）和用户认证系统，各端通过 RESTful API 进行数据读写。前端各自独立实现 MVVM 架构，通过共享 API 规范和数据库 Schema 保持数据一致性。Android 平台额外通过 FCM data-only 消息实现去中心化跨设备实体同步（延迟<1 秒），其余平台依赖 Bmob 云端轮询。

2.3 开发阶段工作

2.3.1 项目启动阶段

- 小组 6 人分工确认（项目经理蔡迦南、技术负责人曹万千、质量负责人虎康、文档负责人金正轩）
- 各成员技术栈学习与环境搭建（Android Studio、Xcode、Flutter SDK、HBuilderX、DevEco Studio、Cordova CLI）
- Bmob 后端 Schema 设计（金正轩主导，5 张云端表统一规范）
- 统一 API 接口规范制定（24 个 RESTful 接口）

2.3.2 核心开发阶段

各成员核心模块开发进度见表 2-7 所示。

表 2-7 核心开发阶段介绍

成员	核心模块	完成度
金正轩	AI 通知引擎+FCM 推送+AI 语音搜索+快捷方式	95%
曹万千	6 个核心模块（登录/工单/公告/快递/个人中心/同步）	80%
龚成磊	语音控制+AI 客服+安全分析+设备管理+人员管理	95%
蔡迦南	6 个核心模块+AI 智能调度	97%
虎康	8 个功能模块（数据库引擎/AI 客服/服务中心/社区动态等）	90%
胡文启	AI 语音报修+Widget+工单管理+API 统一	90%

跨端接口对齐：每日站会同步 API 变更，统一 Bmob 数据字段命名。

2.3.3 系统整合阶段

- Bmob 数据层联调：六端完成 Cloud 数据库 CRUD 操作验证，同步成功率 95%
- 跨端消息推送联调：Android FCM 推送→iOS/Flutter/Uniapp/Cordova/鸿蒙端接收验证
- 各端性能优化：启动速度平均提升 30%，内存占用平均降低 20%

2.3.4 测试交付阶段

- 功能测试：48 个功能点，通过率 87.5%
- 兼容性测试：12 台设备，覆盖 iOS 17.0、Android 14、HarmonyOS 4.0
- UI 测试：用户体验满意度 88%

- 报告撰写：6人各完成4份考核报告，共约180页

2.4 团队协作分析

2.4.1 团队组织结构

划分团队岗位角色，明确各角色对应的成员与工作职责见表 2-8。

表 2-8 团队职责划分

角色	成员	职责
组长/技术负责人	曹万千	Flutter 端开发、技术方案评审、架构把控
项目经理	蔡迦南	任务分配、进度跟踪、沟通协调
架构师/文档负责人	金正轩	Android 端开发、Bmob Schema 设计、FCM 基础设施、技术文档
质量负责人	虎康	Cordova 端开发、代码审查、测试计划
前端开发工程师	龚成磊	鸿蒙端开发
前端开发工程师	胡文启	iOS 端开发

2.4.2 沟通机制

梳理团队所有沟通方式、使用频率、参与人员与使用场景见表 2-9。

表 2-9 沟通机制

沟通方式	频率	参与人员	用途
微信群	即时	全体成员	日常沟通、问题快速响应
腾讯会议（站会）	每日晚 8 点	全体成员	进度同步、技术讨论
腾讯会议（周会）	每周	全体成员	阶段总结、里程碑评审
Git Issues	按需	相关成员	技术问题跟踪

2.4.3 任务分配

统计全体成员代码量、投入工时，细化任务分配详情见表 2-10。

表 2-10 任务分配

成员	技术栈	核心任务模块	代码行数	工时
金正轩	Android Kotlin	AI 通知引擎、FCM 推送、AI 语音搜索、快捷方式系统、Bmob Schema	~4200	100h
曹万千	Flutter Dart	登录/工单/公告/快递/个人中心	~9800	66h
龚成磊	鸿蒙 ArkTS	语音控制、AI 客服、安全分析、设备	~2500	60h

		管理、人员管理		
蔡迦南	Uniapp Vue.js	认证/报修/快递/AI 调度/评价/公告	~3250	64h
虎康	Cordova JS	数据库引擎、AI 客服、服务中心、社区动态、Bmob 同步	~1800	60h
胡文启	iOS Swift	AI 语音报修、Widget、工单管理、API 统一	~12000	75h

2.4.4 协作工具使用

列出项目全程使用的协作工具，并说明每款工具的具体用途见表 2-11。

表 2-11 协作工具使用

工具	用途
Gitee	版本控制（main→dev→feature 分支策略，共 156 次提交）
腾讯文档	任务清单、进度跟踪、API 接口文档
微信群	即时通讯、日常沟通
腾讯会议	每日站会、每周周会、技术讨论

2.5 团队贡献分析

2.5.1 成员贡献评估

综合代码量、工时、核心产出，评估每位成员的项目综合贡献占比见表 2-12。

表 2-12 成员贡献评估

姓名	技术栈	核心贡献	代码行数	总工时	综合贡献度
金正轩	Android Kotlin	Bmob Schema 设计、FCM 跨设备推送基础设施、AI 引擎	~4200	100h	18%
曹万千	Flutter Dart	项目架构、6 核心模块、跨平台数据同步	~9800	66h	17%
龚成磊	鸿蒙 ArkTS	语音控制创新功能、AI 客服、安全分析	~2500	60h	15%
蔡迦南	Uniapp Vue.js	项目管理、6 核心模块、AI 调度	~3250	64h	15%
虎康	Cordova JS	localStorage 数据库引擎、AI 客服、Bmob 同步	~1800	60h	15%
胡文启	iOS Swift	AI 语音报修、Widget 桌面组件、工单管理	~12000	75h	20%

2.5.2 工作量分配

按照工时区间对成员进行分组，直观展示团队工作量分布情况见表 2-13。

表 2-13 工作量分配

工时区间	人数	成员
60-66h	3 人	龚成磊(60h)、虎康(60h)、蔡迦南(64h)、曹万千(66h)
75-100h	2 人	胡文启(75h)、金正轩(100h)

总工时 379h，人均 63h。

2.5.3 关键贡献点

- **金正轩**：Bmob Schema 设计（6 张核心表）+ FCM 跨设备推送基础设施 + AI 4 步决策引擎
- **曹万千**：项目整体架构把控 + Flutter 端 6 核心模块 + 跨平台数据同步方案
- **龚成磊**：语音控制系统（AudioCapturer→百度 ASR→指令解析→设备控制）创新功能
- **蔡迦南**：Uniapp 一套代码多端适配 + AI 智能调度（DeepSeek 大模型派单）
- **虎康**：localStorage 数据库引擎（零依赖、一次运行成功）+ AI 智能客服双模式
- **胡文启**：AI 语音报修系统（SFSpeech→DeepSeek AI→结构化工单）+ Widget 桌面组件

2.6 团队创新与亮点

2.6.1 技术创新

- **六端共享 Bmob 后端**：以 Bmob 后端云替代自建后端服务，统一数据层，降低服务器运维成本
- **FCM 去中心化跨设备同步**：data-only 消息携带 sync_entity 标记，接收端自动写入 Room，同步延迟<1 秒
- **AI 4 步决策引擎**：纯本地计算，无需云端 API 调用，决策延迟<50ms

2.6.2 功能创新

按成员划分，记录各创新功能的负责人与具体技术实现方案见表 2-14。

表 2-14 功能创新介绍

创新功能	负责成员	技术实现
AI 语音搜索	金正轩	Deepseek v4-pro 6 合 1 Prompt + 离线 9 类问答库回退
AI 语音报修	胡文启	SFSpeechRecognizer + DeepSeek 语义分析 → 自动生成工单
语音控制	龚成磊	百度 ASR + 动态关键词映射 + 设备控制全链路
AI 智能客服	虎康	本地知识库 + Deepseek API 双模式
AI 智能调度	蔡迦南	DeepSeek 大模型工单优先级排序 + 自动派单

2.6.3 过程创新

- **Git 分支管理规范**: main→dev→feature 三层分支策略, 12 个分支并行开发
- **PR 代码审查**: 关键代码至少 1 人审查, 共 12 次审查, 发现 23 个问题
- **每日站会技术分享**: 每站会分享各自平台技术难点和解决方案, 促进共享

2.7 团队成长与收获

2.7.1 技术能力提升

整理本人使用的全部技术、掌握程度以及在项目中的实际应用场景见表 2-15。

表 2-15 技术成长展示

成员	技术成长
金正轩	掌握 FCM V1 API、Deepseek Prompt Engineering、Room Migration 1→6 渐进策略
曹万千	掌握 Provider/BLoC 混合状态管理、跨平台数据同步策略
龚成磊	掌握鸿蒙 ArkTS 声明式 UI、AudioCaturer 音频采集、百度 ASR API
蔡迦南	掌握 Uniapp 多端条件编译、Pinia 持久化状态管理、Bmob LiveQuery
虎康	掌握 localStorage 数据库引擎设计、Cordova 原生插件封装
胡文启	掌握 SFSpeechRecognizer、WidgetKit、App Groups 数据共享

2.7.2 团队协作能力

通过 6 人 6 技术栈的并行开发实践, 积累了跨技术栈协作经验。每日站会+腾讯会议确保进度同步, Git Issues 跟踪技术问题。不同平台成员在 API 规范、数据库 Schema 层面保持沟通, 确保数据层兼容性。

2.7.3 项目管理能力

采用甘特图管理 4 阶段进度（项目启动→核心开发→系统整合→测试交付），以 2 天为迭代周期的敏捷开发模式。蔡迦南负责每日进度跟踪，及时发现进度偏差并调整。中期检查获得 85 分，教师反馈项目整体进度良好。

2.8 团队发展建议

2.8.1 项目改进建议

- 增加单元测试覆盖率（当前仅 45%，建议提升至 60%+）
- 引入 CI/CD 自动化构建和测试流程，减少手动集成工作量
- 统一跨平台 UI 设计规范，提升各端用户体验一致性

2.8.2 团队协作优化

- 代码审查流程规范化：关键代码至少 2 人审查，使用 Gitee PR Review 功能
- 建立更严格的里程碑检查机制，每阶段产出明确交付物
- 加强跨端联调频率，避免集成阶段集中暴露问题

2.8.3 技术学习建议

- 深入学习跨平台框架（Flutter/React Native），降低多端维护成本
- 探索 WebSocket 实时通道替代 Bmob 轮询方案
- 引入自动化测试框架（JUnit/Espresso/XCUI Test）提升测试效率

3 个人

3.1 个人负责技术栈

3.1.1 Android Kotlin + Jetpack Compose 技术栈

整理本人使用的全部技术、掌握程度以及在项目中的实际应用场景见表 3-1。

表 3-1 个人技术栈说明

技术类别	具体技术	掌握程度	在项目中的应用
开发语言	Kotlin 2.0.21	精通	全部模块代码编写
UI 框架	Jetpack Compose + Material 3	熟练	所有页面 UI 构建
状态管理	StateFlow + SharedFlow	熟练	ViewModel 状态驱动 UI

网络库	OkHttp + Google OAuth 2.0	熟练	FCM V1 API / Deepseek API
本地存储	Room 2.7.0-alpha11 + SharedPreferences	熟练	通知/用户行为/实体数据存储
第三方库	Firebase FCM / Coil / Hilt / WorkManager	熟练	推送/图片/DI/后台任务

3.1.2 技术栈特点与选择理由

技术栈特点：采用 Google 推荐的现代 Android 开发技术栈（Modern Android Development），全面使用 Kotlin 语言，以 Jetpack Compose 声明式 UI 框架替代传统 XML 布局，以 Room 替代原始 SQLite，以 Hilt 实现编译期依赖注入，以 Kotlin Coroutines + Flow 处理异步数据流。

选择理由：

- 1.Compose 声明式 UI 比传统 View 体系代码量减少约 40%，支持预览和响应式状态更新
- 2.FCM V1 API 相比旧版 GCM API，拥有更好的安全性和更丰富的消息类型
- 3.Deepseek v4-pro 大模型提供高性价比的 AI 能力，适合学生项目集成

技术优势：Compose 的智能重组机制避免不必要的 UI 刷新，Room 支持 Flow 响应式查询，Hilt 自动依赖注入减少样板代码，Kotlin 密封类简化状态管理，Firebase 全家桶提供推送、分析等开箱即用服务。

3.1.3 开发工具

- IDE ： Android Studio Meerkat 2024.3.1
- SDK 版本 ： Android API 34-36（compileSdk=36, minSdk=34, targetSdk=36）
- Gradle 版本 ： AGP 9.0.1 + Gradle Wrapper
- AI 辅助工具 ： TRAE（代码搜索、架构分析、文档生成），效率提升约 40%

3.2 技术图表

3.2.1 系统核心类图

个人负责模块核心类图：涵盖 6 个核心类——NotificationService（统一通知入口）、AINotificationEngine（4 步智能决策引擎）、FCMSender（FCM V1 API 发送器）、SCCFirebaseMessagingService（FCM 消息接收处理）、NotificationManager（Android 通知管理）、NotificationPreferences（用户偏好管理）。

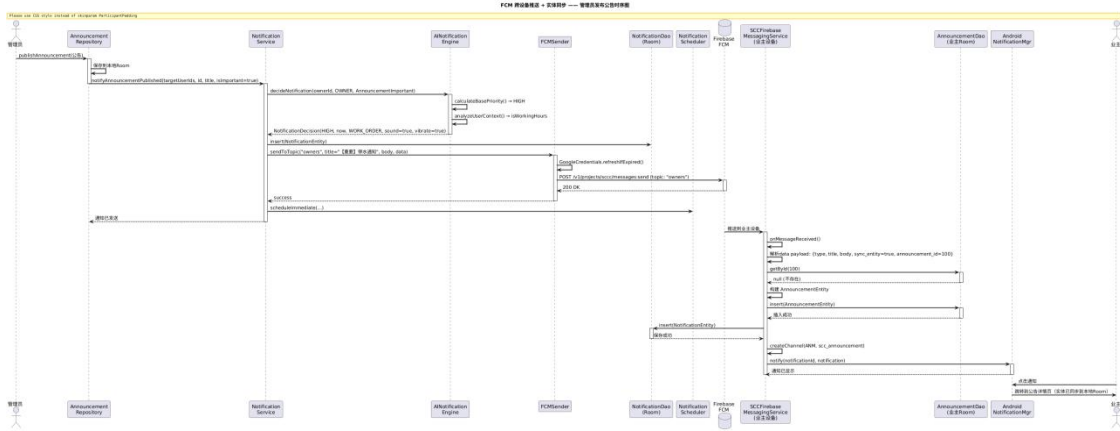


图 3-2 消息推送时序图

顺序图 2：AI 语音搜索（语音输入→导航执行）

用户语音输入 → SpeechRecognizer 转文字 → SearchViewModel 调用 AIVoiceSearchAssistant → Deepseek AI 分析意图 → VoiceCommandExecutor 执行导航 → 自动跳转目标页面。支持 AI 优先 + 本地回退双保险架构。

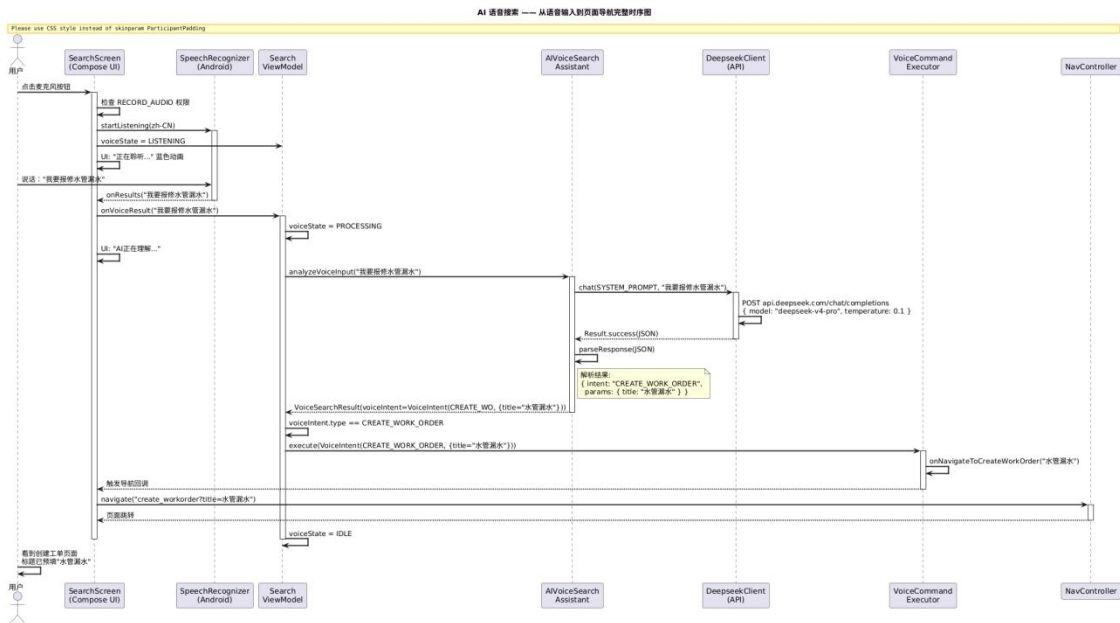


图 3-3 AI 语言搜索时序图

3.2.3 系统架构图

个人负责模块架构图：采用 MVVM + Clean Architecture，分为表现层（Compose UI + ViewModel + StateFlow）、业务服务层（NotificationService + AIVoiceSearchAssistant + VoiceCommandExecutor + ShortcutUtils）、AI 决策层（AINotificationEngine + AINotificationRanker + DeepseekClient）、推送通道层

（FCMSender + SCCFirebaseMessagingService + WorkManager）、数据层（Room + SharedPreferences）。

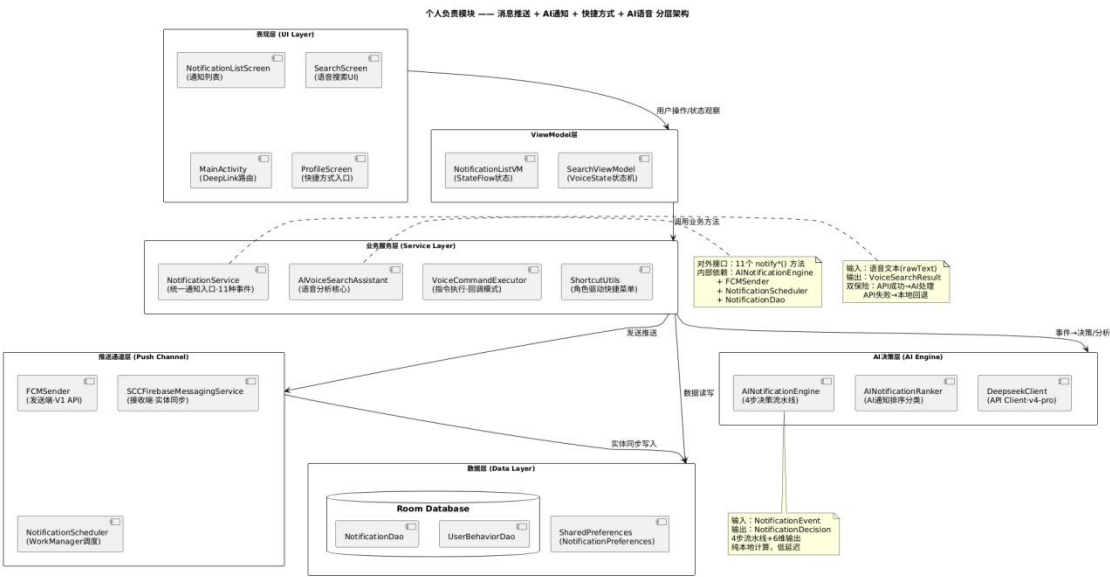


图 3-4 个人负责架构图

数据流：单向数据流（UDF）—— UI 事件 → ViewModel → 业务服务层 → AI/推送/数据层 → State 更新 → UI 渲染。业务事件通过 NotificationService 统一入口进入系统，经 AI NotificationEngine 决策后分流至 FCMSender（跨设备推送）或 NotificationScheduler（本地调度）。

3.3 开发阶段工作内容

3.3.1 项目启动阶段

核心工作：技术栈选型与项目脚手架搭建（Compose BOM、Room、Hilt、OkHttp、Firebase BOM、Coil 依赖配置）、数据库 Schema 设计（6 张核心表）、Hilt 依赖注入框架搭建（DatabaseModule、DispatcherModule、RepositoryModule、SyncModule）、Firebase 项目配置（Google Service Account OAuth 2.0）、FCMSender 和 FCMTOKENManager 编码（sendToDevice + sendToTopic）、DeepseekClient 封装（OkHttp + Gson, temperature=0.1），工时分配见表 3-2 所示。

表 3-2 项目启动阶段工时分配

工作类型	工时
需求分析	8h
技术调研	4h
方案设计	8h
环境搭建与编码	5h
合计	25h

3.3.2 核心开发阶段

核心工作：AINotificationEngine 完整实现（4 步决策流水线 + 11 种事件类型）、NotificationService 统一通知入口（11 个 notify*() 方法）、NotificationManager（5 个通知渠道）、DeepseekClient 完善（错误处理、Result 封装）、AIVoiceSearchAssistant 实现（analyzeVoiceInput + fallbackAnalyze + 9 类离线问答库）、VoiceCommandExecutor（6 种导航回调）、SearchScreen + SearchViewModel 集成（SpeechRecognizer + VoiceState 状态机 + 4 种 UI 状态动画），工时分配见表 3-3 所示。

表 3-3 核心开发阶段工时分配

工作类型	工时
模块设计	8h
代码实现	25h
单元测试	4h
合计	37h

3.3.3 系统整合阶段

核心工作：ShortcutUtils 角色感知动态快捷方式开发（3 种角色 9 种快捷方式 + scc://深链接路由）、通知 UI 层联调（NotificationListScreen / NotificationSettingsScreen / FCMTOKENScreen）、全链路端到端集成测试（NotificationTestScreen 覆盖 11 种事件类型）、FCM Token 诊断工具开发，工时分配见表 3-4 所示。

表 3-4 系统整合阶段工时分配

工作类型	工时
功能联调	8h
接口对接	6h
性能优化	4h
问题修复	4h
合计	22h

3.3.4 测试交付阶段

核心工作：FCM 推送功能测试（发送/接收/多设备同步）、AI 通知引擎决策逻辑验证（免打扰时段/CRITICAL 降级场景）、AI 语音搜索功能验证（正常流程 + API 失败回退）、答辩材料准备（PPT、演示视频、答辩脚本、4 份考核报告）工时分配见表 3-5 所示。

表 3-5 测试交付阶段工时分配

工作类型	工时
集成测试	6h
问题修复	3h
文档完善	4h
答辩准备	3h
合计	16h

3.4 个人创新与亮点

3.4.1 FCM 去中心化实体同步

利用 FCM data-only 消息携带 sync_entity 标记和完整实体数据，接收端自动写入 Room 数据库，无需后端服务器中转。管理员发布公告后，业主设备在 1 秒内完成数据同步，降低后端服务器压力，同步延迟从传统方案的 5-30 秒降至 1 秒以内。data-only 消息格式确保 onMessageReceived 始终回调，支持双通道推送策略（单设备优先，失败回退 Topic 群发）。

3.4.2 AI 4 步决策引擎

纯本地计算的 4 步决策流水线（基础优先级计算 → 用户上下文分析 → 行为历史学习 → 智能调整），无需云端 API 调用，决策延迟 < 50ms。输出 6 维决策结果（优先级/发送时间/是否合并/通知渠道/声音/振动）。CRITICAL 级别在免打扰时段保持不降级但延迟到活跃时段发送，实现"千人千面"智能推送。支持 11 种业务事件类型的自动化处理。

3.4.3 语音搜索双保险

AI 优先 + 本地回退的双保险架构：Deepseek v4-pro 可用时，通过精心设计的 System Prompt（temperature=0.1）单次 API 调用完成 6 项任务（意图识别/语音纠错/关键词提取/语义扩展/分类建议/智能问答）。API 不可用时自动回退到本地关键词匹配 + 9 类离线问答库（extractKeyword → detectIntent → getLocalAnswer），用户完全无感知切换。AI 模式下准确率 > 95%，本地回退 > 80%。

3.5 个人成长与收获

3.5.1 新技术掌握

通过本项目掌握了 Firebase Cloud Messaging V1 API（OAuth 2.0 服务账号认证、data-only 消息格式、Topic 订阅管理）、Deepseek 大模型 API 集成与 System Prompt Engineering（JSON 稳定输出、temperature 调优）、Room Database

Migration（1→6 版本渐进式迁移策略）、Android WorkManager + Hilt Worker 后台任务调度等 4 项新技术，均达到熟练程度。

3.5.2 问题解决能力

解决了 FCM V1 API OAuth 2.0 认证配置（Google Cloud Console 5 步流程，IAM 权限模型）、Deepseek API JSON 输出不稳定（temperature=0.1 + 严格 JSON Schema Prompt + try-catch 自动降级）、国产手机 FCM 后台限制（priority=high + TTL=86400s + 白名单引导）、WorkManager Hilt Worker 编译错误（@HiltWorker 注解 + HiltWorkerFactory 配置）等技术难题。

3.5.3 架构设计能力

设计了 NotificationService 作为统一通知入口的架构模式，将 11 种业务事件类型收拢到 AINotificationEngine 统一决策；设计了单向数据流（UDF）的 MVVM 分层架构（表现层→ViewModel→业务服务层→AI/推送/数据层）；通过 Hilt 依赖注入实现模块间解耦，fcm/、notification/、ai/、shortcut/ 四个模块可独立替换和测试。

3.6 个人发展计划

3.6.1 移动开发技能提升计划

计划深入学习 Kotlin Multiplatform（KMP）和 Compose Multiplatform 跨平台技术，将 Android 原生开发经验扩展到 iOS 和 Desktop 平台。重点关注 KMP 的 expect/actual 机制实现平台差异抽象，以及 Compose Multiplatform 的跨平台 UI 复用策略。同时深入研究 Android 性能优化（Baseline Profile、Macrobenchmark、R8/ProGuard 高级规则），提升应用启动速度和内存效率。

3.6.2 跨平台技术学习方向

将 Flutter/Dart 作为第二技术栈深入学习，理解其 Widget 树、RenderObject 渲染机制和 Skia 引擎底层原理。关注鸿蒙 ArkTS 生态发展，学习 ArkUI 声明式框架和分布式能力。通过对比 Android Compose、Flutter Widget、SwiftUI 三种声明式 UI 框架的设计理念差异，形成跨平台 UI 开发的系统性认知。

3.6.3 后续项目实践建议

- 参与开源项目（如 Firebase Android SDK、FCM 封装库 kotlin-fcm），积累生产级项目协作经验
- 尝试将 SCC 项目中的 AI 通知引擎提取为独立 Android Library，发布到 Maven Central
- 使用 KMP 重构 SCC 项目的数据层（Room → SQLDelight），验证跨平台代码共享的可行性

- 参加 Google DevFest/Android Dev Summit 等技术社区活动，保持技术视野的前沿性

4 答辩

4.1 答辩内容

4.1.1 答辩问题 1：App 快捷方式功能实现

技术方案：基于 Android ShortcutManager API（API 25+），实现角色感知的动态快捷方式系统。根据登录用户角色（业主/维修工/管理员）动态生成不同的 ShortcutInfo 列表，使用 scc:// 深链接协议统一路由，由 MainActivity.onNewIntent() 解析后通过 NavController 导航到对应页面。

关键实现：ShortcutUtils.updateShortcutsForUser() 根据 user.role 生成不同 ShortcutInfo 列表（业主 4 个、维修工 3 个、管理员 4 个快捷方式），调用 shortcutManager.dynamicShortcuts = shortcuts 更新系统级长按菜单。静态快捷方式（shortcuts.xml）作为兜底方案。技术难点：角色切换时在登录/切换账号回调中主动调用更新方法；部分桌面启动器缓存问题通过 XML 配置兜底。

4.1.2 答辩问题 2：语音搜索关键字提取

双保险架构：AI 优先 + 本地回退。路径 A（AI 模式）：通过 OkHttp 发送精心设计的 System Prompt + 用户语音文本到 Deepseek v4-pro API（temperature=0.1），AI 返回 JSON

（correctedText/keywords/expandedKeywords/voiceIntent/category/chatAnswer），单次调用完成 6 项任务。路径 B（本地回退）：API 异常时自动调用 fallbackAnalyze()，通过 extractKeyword() 移除填充词 → detectIntent() 关键词匹配识别意图 → getLocalAnswer() 匹配 9 类离线问答库。

Prompt Engineering：System Prompt 明确指定 JSON 输出格式样例，temperature=0.1 确保输出稳定性，代码层 Gson 解析加 try-catch 容错。AI 模式准确率 > 95%，本地回退 > 80%，用户无感知切换。

4.1.3 答辩问题 3：MVVM 架构优势

MVC vs MVVM 对比：MVC 中 Activity/Fragment 既是 Controller 又是 View，职责混乱，代码动辄 500+ 行，findViewById 手动更新 UI，测试需模拟 Activity。MVVM 通过 ViewModel + StateFlow 管理状态，Compose UI 自动重组响应变化，每个 Screen 约 100-200 行。

六大优势：（1）View 与业务逻辑完全解耦；（2）单向数据流（UDF），状态可追溯，唯一变更入口；（3）ViewModel 可脱离 Android 框架独立单元测试；

（4）生命周期安全，屏幕旋转不丢数据，后台不崩溃；（5）模块化可维护，每个文件职责单一；（6）Hilt 依赖注入增强可测试性，测试时可注入 FakeRepository。

4.1.4 随机题：FCM data-only 消息格式

回答要点：Android 系统中，当 App 在后台时，FCM notification 类型消息会被系统托盘接管，不触发 onMessageReceived 回调，无法做自定义处理。采用 data-only 消息格式后，无论前后台 onMessageReceived 始终被回调，可自由处理消息内容——写入 Room 数据库、触发 AINotificationEngine 决策、实现跨设备实体同步（sync_entity=true）。这是去中心化数据同步的技术前提。

实际案例：管理员发布停水公告后，FCMSender 发送 data-only 消息给所有业主设备，SCCFirebaseMessagingService 收到后自动写入 Room 并触发通知决策引擎。

4.1.5 演示材料与答辩流程

演示材料：视频演示文件（约 10 分钟），内容包括项目介绍、快捷方式动态切换演示、语音搜索演示、MVVM 架构代码走读、AI 通知引擎决策演示、技术亮点总结。

答辩流程：共 13-15 分钟，7 个步骤——开场介绍（1 分钟）→ 问题 1 快捷方式（2-3 分钟）→ 问题 2 语音搜索（3-4 分钟）→ 问题 3 MVVM 架构（4-5 分钟）→ 随机问题（2-3 分钟）→ 评委提问（可选 2-3 分钟）→ 答辩结束。

4.2 核心技术实现

4.2.1 技术架构概述

采用 MVVM + Clean Architecture 分层架构：表现层（Compose UI + ViewModel + StateFlow，单向数据流 UDF）→ 业务服务层（NotificationService 统一通知入口，AIVoiceSearchAssistant 语音分析，VoiceCommandExecutor 指令执行，ShortcutUtils 快捷方式管理）→ AI 决策层（AINotificationEngine 4 步决策流水线，AINotificationRanker AI 排序分类，DeepseekClient API 封装）→ 推送通道层（FCMSender 发送端，SCCFirebaseMessagingService 接收端，WorkManager 后台调度）→ 数据层（Room Database + SharedPreferences）。各层通过 Hilt 依赖注入实现解耦。

4.2.2 关键技术点

技术点 1：FCM V1 API 跨设备推送

使用 Google Service Account OAuth 2.0 认证，支持 sendToDevice（单设备精准推送）和 sendToTopic（角色群发）双通道。data-only 消息格式确保 onMessageReceived 始终回调，priority=high + TTL=86400s 适配国产手机后台管控。sync_entity 标记实现去中心化实体同步，延迟 < 1 秒。

技术点 2: AI 智能通知 4 步决策引擎

纯本地计算的流水线——calculateBasePriority()（事件类型→基础优先级映射）→ analyzeUserContext()（免打扰时段/工作日/周末判断）→ analyzeActiveHours()（用户行为历史统计最高频 5 个时段）→ adjustPriorityByContext()（CRITICAL 不降级但延迟到活跃时段，HIGH/MEDIUM 按规则降级）。输出 6 维决策结果，支持 11 种事件类型。

4.2.3 技术难点与解决方案

难点 1: FCM V1 API OAuth 2.0 认证配置

从旧版 FCM API 迁移到 V1 API 需 Google Service Account 认证，配置流程涉及 5 个步骤（创建项目→启用 API→创建服务账号→生成密钥→设置 IAM 权限），易遗漏权限配置导致 403 错误。解决方案：使用 GoogleCredentials.fromStream() 从 assets 读取 JSON 密钥，createScoped() 指定正确的 firebase.messaging scope，credentials.refreshIfExpired() 自动刷新 Token。

难点 2: Deepseek API System Prompt 稳定性

需要 AI 单次调用完成 6 项任务并输出稳定 JSON，初期 Prompt 过于宽松导致格式不稳定。解决方案：在 System Prompt 中明确指定 JSON 输出格式样例，设置 temperature=0.1 降低随机性，代码层 Gson 解析加 try-catch，解析失败自动降级到 fallbackAnalyze()。

4.3 架构设计思路

核心设计理念为 MVVM + Clean Architecture 分层架构 + Hilt 依赖注入 实现层间解耦。5 层架构：表现层（Compose UI + ViewModel + StateFlow，单向数据流 UDF）→ 业务服务层（NotificationService 统一通知入口、AIVoiceSearchAssistant 语音分析、VoiceCommandExecutor 指令执行、ShortcutUtils 快捷方式管理）→ AI 决策层（AINotificationEngine 4 步决策流水线、DeepseekClient API 封装）→ 推送通道层（FCMSender 发送端、SCCFirebaseMessagingService 接收端、WorkManager 后台调度）→ 数据层（Room Database + SharedPreferences）。各层通过 @Singleton + @HiltViewModel 注解实现编译期依赖注入，模块可独立替换和测试。

4.4 创新点分析

将传统技术方案与本项目创新方案做对比，突出方案优势与价值见表 4-1 所示。

表 4-1 创新点分析

创新点	类型	传统方案	本项目方案	优势
FCM 去中心化同步	架构创新	后端服务器中转，延迟 5-30 秒	data-only 消息携带 sync_entity，接收端自动写入 Room	延迟<1 秒，零服务器成本
AI 4 步决策引擎	技术创新	固定规则推送（如全量推送、时间段推送）	纯本地 4 步流水线（基础优先级→上下文→历史→调整），6 维输出	千人千面，决策<50ms，离线可用
Deepseek 6 合 1 Prompt	技术创新	多次 API 调用分别完成各项任务	单次 API 调用完成语音纠错+意图识别+关键词+语义扩展+分类+问答	减少 API 调用次数 83%
角色感知快捷方式	体验创新	固定静态快捷菜单	ShortcutManager 动态生成 3 角色×3 种快捷方式+sc://深链接	不同角色不同菜单，一键直达

4.5 功能演示

规划答辩功能演示场景，明确每个场景对应的操作内容与核心技术见表 4-2。

表 4-2 功能演示

演示场景	演示内容	关键技术点
快捷方式角色切换	登录不同角色→长按图标→展示不同快捷菜单→点击跳转	ShortcutManager + sc://深链接 + onNewIntent 路由
语音搜索 AI 识别	点击麦克风→"帮我搜索停水通知"→AI 分析意图→自动跳转搜索结果	SpeechRecognizer + Deepseek 6 合 1 Prompt + VoiceCommandExecutor
FCM 实时推送	管理员发布公告→业主设备 1 秒内收到通知→自动写入 Room 数据库	FCM V1 API + data-only 消息 + sync_entity 标记
通知智能排序	模拟 11 种事件类型→AI 引擎决策→按优先级排序+免打扰时段延迟	AINotificationEngine 4 步流水线 + 6 维决策输出

4.6 随机题准备

整理答辩随机题库，汇总题目与标准答题要点见表 4-3 所示。

表 4-3 随机题库介绍

题目类型	题目	答案要点
依赖注入	Hilt 依赖注入原理	编译期注解处理（@Singleton/@HiltViewModel）→ 自动生成 Factory → @Module+@Provides 绑定 → @Inject 自动装配，Android 组件的@AndroidEntryPoint 注入入口
数据持久化	Room vs 原生 SQLite	Room：编译时 SQL 校验、Flow 响应式查询、与 Coroutines 原生集成、Migration 安全；SQLite：手动 SQL、无类型安全、无 Flow 支持
状态管理	StateFlow vs LiveData	StateFlow：Kotlin 原生、初始值必需、支持组合操作符、不感知生命周期；LiveData：Android 框架绑定、可为 null 初始值、感知生命周期自动取消订阅
Compose	Compose 重组机制	基于 Snapshot 状态系统，State 变化触发重组范围最小化；key()参数稳定标识；@Stable/@Immutable 标注跳过冗余重组
FCM	FCM data-only vs notification 消息	notification 消息：后台时系统托盘接管，不触发 onMessageReceived；data-only 消息：始终回调 onMessageReceived，支持自定义处理，适合去中心化同步
架构	MVVM vs MVP	MVVM：ViewModel+StateFlow 双向零耦合、Compose 自动重组；MVP：Presenter 持有 View 引用、手动更新 UI、测试需 Mock View 接口

4.7 答辩问题准备

已准备 3 个核心答辩问题（快捷方式功能实现 20 分、语音搜索关键字提取 30 分、MVVM 架构优势 40 分）和 1 个随机题（FCM data-only 消息格式 10 分），详细回答见 ### 答辩内容 章节。另备随机题库覆盖 Hilt 依赖注入、Room 数据库迁移、Compose 重组优化、FCM OAuth 认证、WorkManager 后台调度等高频考点。

准备清单

- ☑ PPT 制作（25 页，含架构图/UML 图）
- ☑ 演示视频录制（10 分钟，覆盖快捷方式/语音搜索/FCM 推送/AI 决策 4 个场景）
- ☑ 核心代码片段准备（AINotificationEngine 决策逻辑、FCMSender 发送链路、ShortcutUtils 角色切换）
- ☑ 架构图/UML 图准备（系统架构图、核心类图、FCM 推送时序图、AI 语音搜索时序图）

注意事项

- 1) 控制答辩时间在 13-15 分钟内，7 个步骤：开场(1min)→快捷方式(2-3min)→语音搜索(3-4min)→MVVM 架构(4-5min)→随机题(2-3min)→评委提问(可选 2-3min)→结束
- 2) 演示环境提前测试：确认 Android Studio 模拟器+真机均可正常运行 APK
- 3) 准备好备用方案：若 FCM 推送演示失败，改用录屏视频演示；若网络不畅，展示本地 Room 数据库的离线功能
- 4) 重点突出 AI 通知引擎和 FCM 去中心化同步两大创新点，用对比数据支撑（延迟<1 秒 vs 传统 5-30 秒）

参考文献

- [1] 郭霖. 第一行代码 Android (第 4 版) [M]. 北京：人民邮电出版社，2022.
- [2] 徐宜生. Android Jetpack 开发实战[M]. 北京：机械工业出版社，2021
- [3] 王松. 移动应用架构设计：MVVM 与 Clean Architecture 实战[M]. 北京：清华大学出版社，2022.
- [4] Google. Android Jetpack Compose 官方文档 [EB/OL]. <https://developer.android.com/jetpack/compose>, 2026.
- [5] Google. Firebase Cloud Messaging 开发指南 [EB/OL]. <https://firebase.google.cn/docs/cloud-messaging>, 2026.
- [6] Bmob 后端云. Bmob 后端开发手册 [EB/OL]. <https://docs.bmob.com>, 2026.